

TALLER 2: BÚSQUEDA LOCAL Y JUEGOS ADVERSARIOS

CONFIGURACIÓN INICIAL MEDIANTE ASCENSO DE COLINA

Formulación del Problema de Optimización

Elemento	Definición en el Proyecto
Representación	Tupla binaria $(x_1, \dots, x_n)$; $x_i = 1$ indica que el sitio candidato i tiene un módulo instalado
Restricción de Validez	Exactamente k bits activos (<code>problem.is_valid</code> , $k = \text{module_count}$)
Función Objetivo	puntaje = cobertura – redundancia – exposición, calculada por <code>configuration_score</code> a partir de <code>problem.score_components</code>
Relación de Vecindad	Retirar el módulo de un sitio activo y ubicarlo en uno inactivo (<code>problem.neighbors</code>); conserva k módulos

Implementación de `configuration_score` y `hill_climbing`

Se implementó `configuration_score` utilizando: puntaje = cobertura – redundancia – exposición, donde cada componente es retornado por `problem.score_components(configuration)`.

El algoritmo `hill_climbing` implementa mejora estricta: examina todos los vecinos, avanza al de mayor puntaje solo si es estrictamente mejor, se detiene cuando no existe mejora o se alcanza el límite de iteraciones.

Se ejecutó `hill_climbing` con tres semillas distintas en cada layout:

Layout	Semilla	Mejor Puntaje	Evaluaciones	Análisis
local_peak	7	96.00	46	Óptimo global encontrado
local_peak	3	96.00	31	Óptimo global encontrado
local_peak	12	81.00	31	Máximo local (15 puntos inferior)
plateau_grid	7	92.00	31	Mesetas presentes, búsqueda larga
plateau_grid	12	92.00	46	Resultado similar, distinto costo
plateau_grid	3	94.00	31	Buen desempeño

En `local_peak`, la semilla 12 parte de una configuración inicial desde la cual todos los vecinos inmediatos son iguales o peores, por lo que el algoritmo se detiene en 81.00 un máximo local claramente inferior al 96.00 alcanzado con las semillas 3 y 7. Esto confirma que la calidad de la solución de ascenso de colina depende fuertemente de la inicialización: no hay mecanismo de escape una vez que ninguna mejora estricta está disponible.

En `plateau_grid` las tres semillas llegan a valores cercanos (92–94) pero con distinto número de evaluaciones, lo que sugiere la presencia de mesetas (regiones de puntaje similar) que alargan la búsqueda sin necesariamente mejorar el resultado final.

ESCAPE DE ÓPTIMOS LOCALES MEDIANTE RECOCIDO SIMULADO

El recocido simulado busca superar los máximos locales aceptando temporalmente movimientos que reducen el puntaje. Se implementó `cooling_schedule` con el programa geométrico $T(t) = T_0 \times \alpha^t$ y `simulated_annealing` que acepta mejoras siempre y movimientos peores con probabilidad $\exp(\Delta E / T)$.

c) Comparación de Configuraciones (Layout: `risk_corridor`, 5 ejecuciones por configuración, semilla base 7)

Algoritmo	Parámetros	Mejor	Promedio	Desviación	Eval. Prom.	Característica
HC	—	116.00	110.60	3.38	153	Rápido, variable
SA	$T_0=5, \alpha=0.90$	109.00	108.60	0.49	213	Óptimo local
SA	$T_0=20, \alpha=0.97$	116.00	113.40	2.58	501	Intermedio
SA	$T_0=50, \alpha=0.995$	116.00	115.60	0.80	501	✓ Óptimo

Con temperatura inicial baja y enfriamiento rápido ($T_0=5$, $\text{cooling}=0.90$), el algoritmo casi no acepta movimientos que empeoran el puntaje: converge rápido (213 evaluaciones) pero queda atrapado cerca de un óptimo local inferior (108.60 en promedio), con baja varianza porque explora poco.

Con temperatura inicial alta y enfriamiento lento ($T_0=50$, $\text{cooling}=0.995$), el algoritmo acepta muchos movimientos deteriorantes al inicio, explora más regiones del espacio y termina con el mejor promedio (115.60) y menor varianza que la configuración intermedia, a costa de usar el límite completo de iteraciones (501 evaluaciones, más del doble que la configuración rápida).

Frente a ascenso de colina, SA bien calibrado ($T_0=50$) mejora tanto el promedio (115.60 vs 110.60) como la consistencia (desviación 0.80 vs 3.38), confirmando que aceptar temporalmente el deterioro permite escapar de los máximos locales donde hill climbing se estanca.

OPTIMIZACIÓN POBLACIONAL MEDIANTE ALGORITMO GENÉTICO

Se implementó el algoritmo genético integrando evaluación, selección por torneo, cruce (`one_point_crossover`), mutación (`swap_mutation`), reparación y elitismo para mantener los mejores individuos.

Comparación de Configuraciones (Layout: `balanced_city`, 5 ejecuciones por configuración, semilla base 7)

Pop	Gen	Mut	Élite	Mejor	Promedio	Desv.	Eval.	Observación
40	100	0.05	2	283	282.20	0.98	4040	Baseline
10	100	0.01	1	283	279.60	2.06	1010	Poca diversidad
40	100	0.30	2	283	281.80	0.98	4040	Mutación alta

Efecto del tamaño de población:

Al reducir la población de 40 a 10 individuos, el costo cae proporcionalmente (=4000 a =1000 evaluaciones), pero la calidad se vuelve menos consistente entre corridas:

la desviación estándar sube de 0.98 a 2.06. Con menos individuos hay menor diversidad genética, por lo que algunas corridas quedan atrapadas en óptimos inferiores antes de agotar las generaciones.

Efecto de la probabilidad de mutación:

Subir la mutación de 0.05 a 0.30 no mejora el resultado (281.80 vs 282.20 en promedio) e incluso lo empeora levemente: una mutación demasiado alta introduce ruido excesivo en los descendientes, deshaciendo combinaciones útiles logradas por el cruce sin aportar exploración útil adicional en este layout.

En los tres casos el elitismo garantiza que el mejor puntaje (283.00) se alcanza siempre, confirmando que `genetic_algorithm` conserva el mejor individuo global independientemente de la variabilidad entre generaciones.

DEFENSA ADVERSARIA MEDIANTE MINIMAX

Formulación Formal del Juego

Componente	Definición en el Proyecto
Estado Inicial	<code>GameState.initial(layout)</code> : Defensor en D, Intruso en I, todas terminales pendientes
Jugador en Turno	Agente 0 (MAX) = Defensor, Agente 1 (MIN) = Intruso; alternan en cada ply
Acciones	<code>get_legal_actions(agent_index)</code> : North, South, East, West, Stop (filtradas por bloqueos)
Función de Sucesor	<code>generate_successor(agent_index, action)</code> : mueve agente, activa terminal si defensor la alcanza
Prueba Terminal	<code>is_win()</code> (sin terminales pendientes) / <code>is_lose()</code> (defensor e intruso misma posición)
Utilidad	+1000 en victoria, -1000 en derrota (<code>base_evaluation_function</code>)

Experimentos de Profundidad (Control: `single_terminal`, Comparación 2 vs 4 plies)

Layout	Prof	Acción	Resultado	Nodos	Análisis
<code>single_terminal</code>	2	East	Victoria	112	Control OK comportamiento estable
<code>tiny_defense</code>	2	North	Victoria	260	Buen desempeño con profundidad 2
<code>tiny_defense</code>	4	South	Victoria	4110	15.8x más nodos que prof 2
<code>bottleneck</code>	2	Stop	Derrota	27	Horizonte insuficiente: no ve escape
<code>bottleneck</code>	4	West	Victoria	2192	Profundidad resuelve el problema
<code>deceptive_distance</code>	2	West	Límite	982	Prof insuficiente para conclusión
<code>deceptive_distance</code>	4	North	Victoria	4437	Prof 4 alcanza solución

El control `single_terminal` confirma un comportamiento correcto y estable con un solo objetivo. En `bottleneck`, la profundidad cambia radicalmente la decisión: con 2 plies el defensor no anticipa la maniobra del intruso y elige quedarse quieto (Stop),

lo que termina en derrota; con 4 plies anticipa la secuencia completa y escoge West, ganando la partida. En los tres layouts, aumentar la profundidad de 2 a 4 multiplica los nodos explorados entre 15x y 80x, evidenciando el costo exponencial de anticipar más acciones.

PODA ALFA-BETA Y FUNCIÓN DE EVALUACIÓN

Comparación Minimax vs AlphaBeta (dual_terminal y medium_defense)

Layout	Prof	Acción	Nodos MM	Nodos AB	Reducción	Igual Acción
dual_terminal	2	Stop	432	311	28%	Sí
dual_terminal	4	West	19908	9782	51%	Sí
medium_defense	2	East	737	536	27%	Sí
medium_defense	4	North	12603	4907	61%	Sí

En los cuatro casos, AlphaBetaAgent selecciona exactamente la misma acción inicial que MinimaxAgent, confirmando que la poda no altera la decisión. La reducción de nodos crece con la profundidad (28–27% en 2 plies, 51–61% en 4 plies), porque a mayor profundidad hay más ramas que superan el límite alpha/beta establecido por acciones ya exploradas.

evaluation_function combina, para estados no terminales:

1. Puntaje acumulado (state.get_score()): progreso real de la partida (penaliza movimientos, premia activaciones).
2. Terminales activadas vs. pendientes (+35 por activada, -20 por pendiente): favorece estados donde el defensor ya completó más objetivos.
3. Distancia a la terminal pendiente más cercana ($-18 \times \text{distancia}$, o -120 si es inaccesible): guía al defensor hacia el siguiente objetivo alcanzable.
4. Distancia al intruso ($+\min(12 \times \text{distancia}, 120)$, con penalización de -180 si la distancia ≤ 1): premia mantenerse lejos del intruso y castiga fuertemente el riesgo inmediato de captura.
5. Movilidad ($+4 \times \text{acciones legales del defensor}$): evita estados casi bloqueados sin opciones de escape.

El resultado se acota a $[-999, 999]$, estrictamente dentro del rango no terminal exigido, de modo que ninguna combinación de heurísticas puede igualar o superar las utilidades terminales (+ o -1000). Todos los términos están formulados desde la perspectiva de MAX (el defensor): suman cuando el estado favorece al defensor y restan cuando lo favorece al intruso, por lo que la función distingue de forma coherente estados no terminales favorables (cerca del objetivo, lejos del intruso, con movilidad) de los desfavorables (lejos del objetivo, en riesgo de captura, acorralado).

REFLEXIÓN SOBRE EL TRABAJO REALIZADO

. Análisis de los Algoritmos y Resultados

¿Qué características de los layouts explican las diferencias observadas?

Layouts con máximos locales pronunciados (local_peak) producen los mayores contrastes en ascenso de colina según la inicialización (81 vs 96, diferencia de 15 puntos según semilla), porque una vez que ningún vecino mejora, el algoritmo no

tiene forma de escapar. Layouts con regiones de puntaje similar (plateau_grid) alargan la búsqueda sin cambiar mucho el resultado final. En risk_corridor, con múltiples configuraciones casi equivalentes, el recocido simulado y el algoritmo genético aprovechan su capacidad de explorar varias regiones para superar consistentemente a hill climbing.

¿Qué efecto tuvieron la inicialización, la aleatoriedad y los parámetros?

En ascenso de colina la inicialización es determinante: mismas instancias, distinta semilla, resultados hasta 15 puntos distintos. En recocido simulado, temperatura inicial y factor de enfriamiento controlan el balance exploración/explotación: parámetros conservadores ($T_0=5$, cooling=0.90) convergen rápido a soluciones inferiores y estables; parámetros generosos ($T_0=50$, cooling=0.995) exploran más y logran mejor promedio a mayor costo. En el algoritmo genético, reducir la población de 40 a 10 individuos aumenta la varianza entre corridas (0.98 a 2.06) por pérdida de diversidad genética, y una mutación excesiva (0.30) no mejora el resultado.

¿Qué compromiso existe entre calidad y costo computacional?

Ascenso de colina es el más barato (=150 evaluaciones) pero el más expuesto a óptimos locales. Recocido simulado bien calibrado casi triplica el costo (=500 evaluaciones) a cambio de mejor calidad y menor varianza. El algoritmo genético es el más costoso en términos absolutos (=4000 evaluaciones con población 40) pero paraleliza la exploración en múltiples individuos y garantiza, mediante elitismo, nunca perder la mejor solución encontrada.

Caso donde la profundidad cambia la decisión (efecto horizonte)

En bottleneck, Minimax con profundidad 2 elige Stop y el defensor termina capturado (derrota), mientras que con profundidad 4 elige West y gana la partida. Con solo 2 plies el defensor no alcanza a ver más allá del horizonte de búsqueda la maniobra de escape necesaria, y evaluation_function puntúa el estado inmediato como si quedarse quieto fuera seguro; con 4 plies el árbol sí alcanza a representar la secuencia completa de evasión y la decisión cambia radicalmente. Esto ilustra el efecto horizonte: una heurística razonable en el corte no sustituye la información que solo una búsqueda más profunda puede aportar.

¿Por qué alfa-beta produce la misma decisión que Minimax?

Alfa-beta solo omite ramas cuyo valor ya no puede influir en la decisión del nodo padre: si un valor explorado supera beta (en MAX) o cae bajo alpha (en MIN), el resto de esa rama queda fuera del rango que el padre podría usar, sin importar su valor exacto. Por eso nunca descarta la rama óptima, solo trabajo redundante lo confirman los cuatro experimentos del punto 5c, donde ambos agentes eligen siempre la misma acción. La profundidad incrementa el costo de forma exponencial en ambos algoritmos (factor de ramificación de hasta 5 acciones por agente), pero el orden de las acciones determina cuánta poda es efectiva: en nuestros experimentos, la reducción de nodos pasó de 27-28% en profundidad 2 a 51-61% en profundidad 4, porque examinar primero acciones prometedoras genera cortes más tempranos y profundos; un orden desfavorable (peores acciones primero) reduciría notablemente esa ganancia.

Reflexión sobre el Uso de Inteligencia Artificial

En el desarrollo de este taller se utilizó IA de forma selectiva y estratégica en dos fases clave. En primer lugar, al inicio del proyecto, se empleó para comprender la estructura general del taller y mapear cómo los conceptos teóricos vistos en clase

(búsqueda local, juegos adversarios, poda alfa-beta) se traducían en el código del proyecto entregado. Esto permitió acelerar la curva de aprendizaje sobre la arquitectura del framework RedCentinela sin saltar pasos conceptuales. En segundo lugar, una vez completada la implementación, se utilizó IA de forma iterativa para generar y ejecutar pruebas exhaustivas, verificar el manejo de errores en casos límite y validar que cada componente del código funcionara de manera robusta. Mediante este uso de pruebas asistidas, se detectaron y corrigieron comportamientos inesperados en la lectura de estados, la poda de ramas en alfa-beta y la evaluación de funciones en estados no terminales. En ambas fases, la IA sirvió como herramienta de apoyo para la verificación y el entendimiento, nunca como generadora de la lógica algorítmica principal, que fue desarrollada de forma autónoma. Los cambios sugeridos fueron mayormente mejoras secundarias en robustez; la estructura y decisiones algorítmicas fundamentales permanecen siendo nuestras. Se podría explicar sin dificultad la solución completa en una sustentación, pues cada componente fue revisado, probado y comprendido en profundidad.

CONCLUSIONES GENERALES

- Hill climbing es rápido pero frágil frente a máximos locales; depende críticamente de inicialización.
- Recocido simulado bien calibrado ($T_0=50$, $\alpha=0.995$) escapa óptimos locales y mejora HC en promedio 1.04x (115.60 vs 110.60) con varianza 4.2x menor (0.80 vs 3.38).
- Algoritmo genético garantiza elitismo pero requiere 27x más evaluaciones que HC (4000 vs 150).
- Minimax con $\text{prof}=4$ anticipa 2192 nodos en bottleneck; $\text{prof}=2$ falla por horizonte limitado (decisión: Stop vs West).
- Alfa-beta reduce nodos 28–61% según profundidad, mantiene decisión Minimax, aprovecha orden prometedor de acciones.
- `evaluation_function()` balancea objetivos (terminales), seguridad (distancia intruso) y navegación (movilidad), confinado a $[-999, +999]$.

Todos los algoritmos operan según especificación. Los parámetros elegidos y la topología de los layouts determinan de manera decisiva el desempeño observado.